

Symbol Table

1. What is symbol table and what kind of information it stores? Discuss its capabilities and also explain the uses of symbol table.

Ans: A **symbol table** is a compile time data structure that is used by the compiler to collect and use information about the source program constructs, such as variables, constants, functions, etc. The symbol table helps the compiler in determining and verifying the semantics of given source program. The information in the symbol table is entered in the lexical analysis and syntax analysis phase, however, is used in later phases of compiler (semantic analysis, intermediate code generation, code optimization, and code generation). Intuitively, a symbol table maps names into declarations (called attributes), for example, mapping a variable name `a` to its data type `char`.

Each time a name is encountered in the source program, the compiler searches it in the symbol table. If the compiler finds a new name or new information about an existing name, it modifies the symbol table. Thus, an efficient mechanism must be provided for retrieving the information stored in the table as well as for adding new information to the table. The entries in the symbol table consists of (name, information) pair. For example, for the following variable declaration statement,

```
char a;
```

The symbol table entry contains the name of the variable along with its data type.

More specifically, the symbol table contains the following information:

- ❑ The character string (or lexeme) for the name. If the same name is assigned to two or more identifiers which are used in different blocks or procedures, then an identification of the block or procedure to which this name belongs to must also be stored in the symbol table.
- ❑ For each type name, the type definition is stored in the symbol table.
- ❑ For each variable name, its type (`int`, `char`, or `real`), its form (label, simple variable, or array), and its location in the memory must also be stored. If the variable is an array, then some other attributes such as its dimensions, and its upper and lower limits along each dimension are also stored. Other attributes such as storage class specifier, offset in activation record, etc. can also be stored.
- ❑ For each function and procedure, the symbol table contains its formal parameter list and its return type.
- ❑ For each formal parameter, its name, type and type of passing (by value or by reference) is also stored.

A symbol table must have the following capabilities:

- ❑ **Lookup:** To determine whether a given name is in the table.
- ❑ **Insert:** To add a new name (a new entry) to the table.
- ❑ **Access:** To access the information related with the given name.
- ❑ **Modify:** To add new information about a known name.
- ❑ **Delete:** To delete a name or group of names from the table.

The information stored in the symbol table can be used during several stages of compilation process as discussed below:

- ❑ In semantic analysis, it is used for checking the usage of names that are consistent with respect to their implicit and explicit declaration.
- ❑ During code generation, it can be used for determining how much and what kind of runtime storage must be allocated to a name.
- ❑ The information in the symbol table also helps in error detection and recovery. For example, we can determine whether a particular error message has been displayed before, and if already displayed then avoid displaying it again.

2. What are the symbol table requirements? What are the demerits in the uniform structure of symbol table?

Ans: The basic requirements of a symbol table are as follows:

- ❑ **Structural flexibility:** Based on the usage of identifier, the symbol table entries must contain all the necessary information.
- ❑ **Fast lookup/search:** The table lookup/search depends on the implementation of the symbol table and the speed of the search should be as fast as possible.
- ❑ **Efficient utilization of space:** The symbol must be able to grow or shrink dynamically for an efficient usage of space.
- ❑ **Ability to handle language characteristics:** The characteristic of a language such as scoping and implicit declaration needs to be handled.

Demerits in Uniform Structure of Symbol Table:

- ❑ The uniform structure cannot handle a name whose length exceed upper bound or limit of name field.
- ❑ If the length of a name is small, then the remaining space is wasted.

3. Write down the operations performed on a symbol table.

Ans: The following operations can be performed on a symbol table:

- ❑ **Insert:** The insert operation inserts a new name into the table and returns an index of new entry. The syntax of insert function is as follows:

```
insert(String key, Object binding)
```

For example, the function `insert(s, t)` inserts a new string `s` in the table and returns an index of new entry for string `s` and token `t`.

- ❑ **Lookup:** This operation searches the symbol table for a given name. The syntax of lookup function is as follows:

```
object_lookup(string key)
```

For example, the function `object_lookup(s)` returns an index of the entry for the string `s`; if `s` is not found, it returns 0.

- ❑ **Search/Insert:** This operation searches for a given name in the symbol table, and if not found, it inserts it into the table.
- ❑ **begin_scope () and end_scope ():** The `begin_scope ()` begins a new scope, when a new block starts, that is, when the token `{` is encountered. The `end_scope ()` removes the scope when the scope terminates, that is, when the token `}` is encountered. After removing a scope, all the declarations inside this scope are also removed.
- ❑ **Handling reserved keywords:** Reserved keywords like ‘PLUS’, ‘MINUS’, ‘MUL’, etc., are handled by the symbol table in the following manner.

```
insert ("PLUS", PLUS);
insert ("MINUS", MINUS);
insert ("MUL", MUL);
```

The first ‘PLUS’, ‘MINUS’, and ‘MUL’ in the `insert` operation indicate lexeme and second one indicate the token.

4. Explain symbol table implementation.

Ans: The implementation of a symbol table needs a particular data structure, depending upon the symbol table specifications. Figure 10.1 shows the data structure for implementation of a symbol table.

The character string forming an identifier is stored in a separate array `arr_lexeme`. Each string is terminated by an EOS (end of string character), which is not a part of identifiers. Each entry in the symbol table `arr_symbol_table` is a record having two or more fields, where first field named `lexeme_pointer`

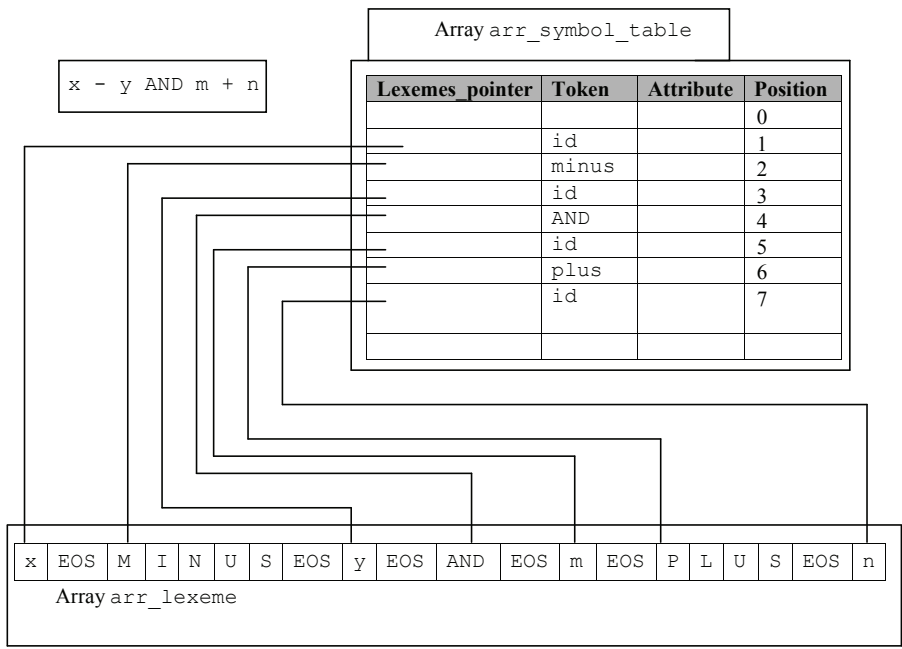


Figure 10.1 Implementation of Symbol Table

points to the beginning of the lexeme, and the second field `Token` consists of the token name. Symbol table also contains two more fields, namely `attribute`, which holds the attribute values, and `position`, which indicates the position of a lexeme in the symbol table are used.

Note that the 0th entry in the symbol table is left empty, as a lookup operation returns 0, if the symbol table does not have an entry for a particular string. The 1st, 3rd, 5th and 7th entries are for the `x`, `y`, `m`, and `n` respectively. The 2nd, 4th and 6th entries are reserved keyword entries for `MINUS`, `AND` and `PLUS` respectively.

Whenever the lexical analyzer encounters a letter in an input string, it starts storing the subsequent letters or digits in a buffer named `lex_buffer`. It then scans the symbol table using the `object_lookup()` operation to determine whether the collected string is in the symbol table. If the lookup operation returns 0, that is, there is no entry for the string in `lex_buffer`, a new entry for the identifier is created using `insert()`. After the insert operation, the index `n` of symbol table entry for the entered string is passed to the parser by setting the `tokenval` to `n`, and an entry in the `Token` field of the token is returned.

5. Discuss various approaches used for organization of symbol table.

Or

Explain the various data structure used for implementing the symbol table.

Ans: The various data structures used for implementing the symbol table are linear list, self-organizing list, hash table, and search tree. The organization of symbol table depends on the selection of the data structure scheme used to implement the symbol table. The data structure schemes are evaluated on the basis of access time, simplicity, storage and performance.

- **Linear list:** A linear list of records is the simplest data structure and it is easiest-to-implement data structure as compared to other data structures for organizing a symbol table. A single array or collection of several arrays is used to store names and their associated information. It uses a simple linear linked list to arrange the names sequentially in the memory.

The new names are added to the table in the order of their arrival. Whenever a new name is added, the whole table is searched linearly or sequentially to check whether the name is already present in the symbol table or not. If not, then a record for the new name is created and added to the linear list at a location pointed to by the `space` pointer, and the pointer is incremented to point to the next empty location (See Figure 10.2).

Variable	Information (type)	Space (byte)
a	int	2
b	char	1
c	float	4
d	long	4

Space

Figure 10.2 Symbol Table as a Linear List

To access a particular name, the whole table is searched sequentially from its beginning until it is found. For a symbol table having n entries, it will take on average $n/2$ comparisons to find a particular name.

- ❑ **Self-organizing list:** We can reduce the time of searching the symbol table at the cost of a little extra space by adding an additional `LINK` field to each record or to each array index. Now, we search the list in the order indicated by links. A new name is inserted at a location pointed to by `space` pointer, and then all other existing links are managed accordingly. A self-organizing list is shown in Figure 10.3, where the attributes `id1` is related to `id2` and `id3` is related to `id1`, and are linked by the `LINK` pointer.

Variable	Information
<code>id₁</code>	Info 1
<code>id₂</code>	Info 2
<code>id₃</code>	Info 3

Figure 10.3 Symbol Table as Self Organizing List

The main reason for using the self-organizing list is that if a small set of names is heavily used in a section of program, then these names can be placed at the top while that section is being processed by the compiler. However, if references are random, then the self-organizing list will cost more time and space.

Demerits of self-organizing list are as follows:

- It is difficult to maintain the list if a large set of names is frequently used.
- It occupies more memory as it has a `LINK` field for each record.
- As self-organizing list organizes it itself, so it may cause problems in pointer movements.

- ❑ **Hash Table:** A hash table is a data structure that associates keys with values. The basic hashing scheme has two parts:
- A hash table consisting of a fixed array of k pointers to table entries.
 - A storage table with the table entries organized into k separate linked lists and each record in the symbol table appears on exactly one of these lists.

To add a name in the symbol table, we need to determine the hash value of that name with the help of a hash function, which maps the name to the symbol table by assigning an integer between 0 to $k - 1$. To search a given name into the symbol, a hash function is applied to that name. Thus, we need to search only that list to determine whether that name exists in the symbol table or not. There is no need to search the entire symbol table. If the name is not present in the list, we create a new record for that name and then insert that record at the head of the list whose index is computed by applying the hash function to the name.

A hash function should be chosen in such a way that it distributes the names uniformly among the k lists, and it can be computed easily for the names comprising character strings. The main advantage of using hash table is that, we can insert or delete any name in $O(n)$ time and search any name in $O(1)$ time. However, in the worst case it can be as bad as $O(n)$.

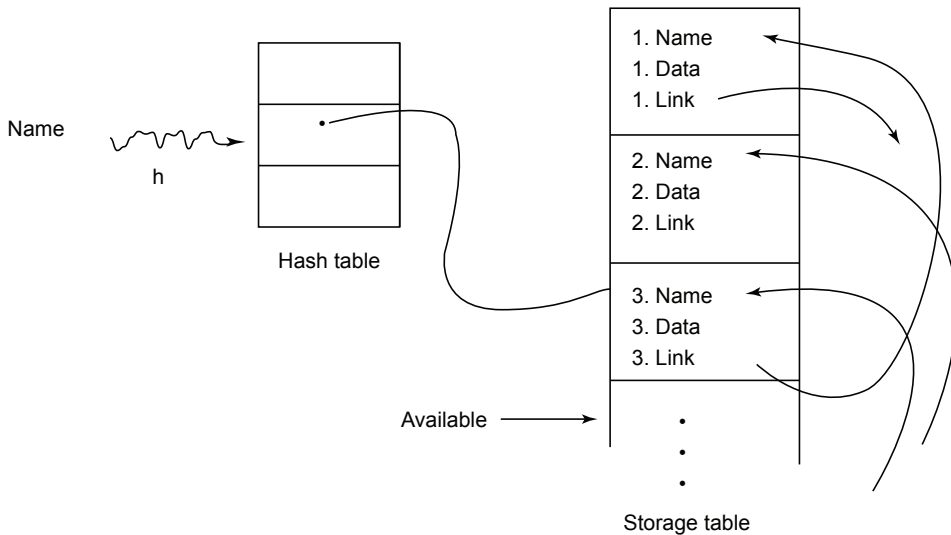


Figure 10.4 Symbol Table as Hash Table

❑ **Search Tree:** Search tree is an approach to organize symbol table by adding two link fields, LEFT and RIGHT, to each record. These two fields are used to link the records into a binary search tree. All names are created as child nodes of root node that always follow the properties of a binary search tree.

- The name in each node is a key value, that is, no two nodes can have identical names.
- The names in the nodes of left sub tree, if exists, is smaller than the value in the root node.
- The names in the nodes of right sub tree, if exists, is greater than the value in the root node.
- The left and right sub trees, if exists, are also binary search trees.

For example: $\text{name} < \text{name_i}$ and $\text{name_i} < \text{name}$. These two statements show that all name smaller than name_i must be left child of name_i ; and all name greater than name_i must be right child of name_i . To insert, search and delete in search tree, the binary search tree insert, search and deletion algorithms are followed respectively.

6. Create list, search tree and hash table for given program.

```
int i, j, k;
int mul (int a, int b)
{
    i = a * b;
    Return (i)
}
main ()
{
    int x;
    x = mul (2, 3);
}
```

Ans: List

Variable	Information	Space
x	integer	2 bytes
i	integer	2 bytes
j	integer	2 bytes
k	integer	2 bytes
a	integer	2 bytes
b	integer	2 bytes
mul	integer	2 bytes
		← Space

Figure 10.5 List Symbol Table for Given Program

Hash Table

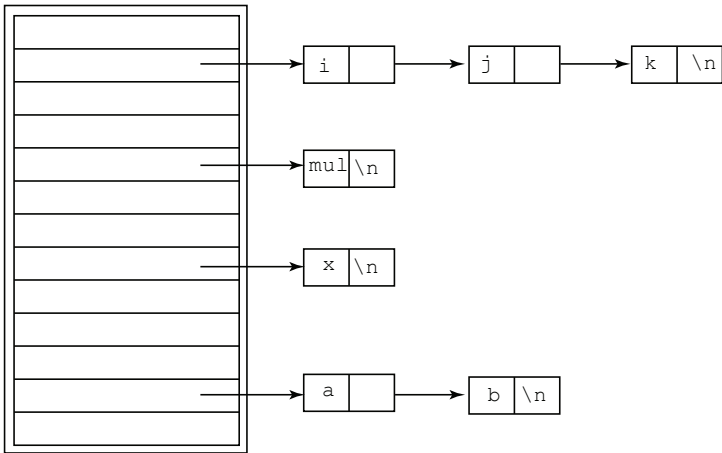


Figure 10.6 Hash Symbol Table for Given Program

Search Tree

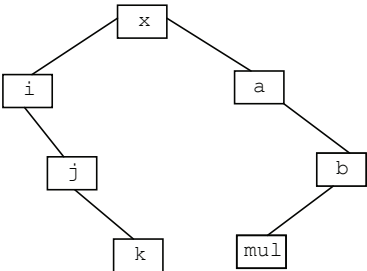


Figure 10.7 Search Tree Symbol Table for the Given Program

7. Discuss how the scope information is represented in a symbol table.

Ans: Scope information characterizes the declaration of identifiers and the portions of the program where it is allowed to use each identifier. Different languages have different scopes for declarations. For example, in FORTRAN, the scope of a name is a single subroutine, whereas in ALGOL, the scope of a name is the section or procedure in which it is declared. Thus, the same identifier may be declared several times as distinct names, with different attributes, and with different intended storage locations. The symbol table is thus responsible for keeping different declaration of the same identifier distinct.

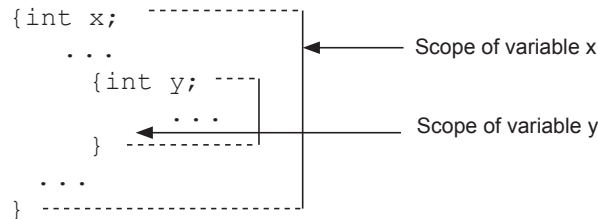
To make distinction among the declarations, a unique number is assigned to each program element that in return may have its own local data. Semantic rules associated with productions that can recognize the beginning and ending of a subprogram are used to compute the number of currently active subprograms.

There are mainly two semantic rules regarding the scope of an identifier.

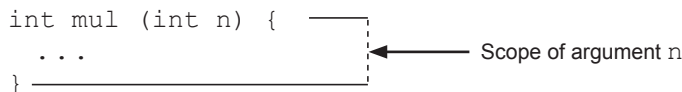
- ❑ Each identifier can only be used within its scope.
- ❑ Two or more identifiers with same name and are of same kind cannot be declared within the same lexical scope.

The scope declaration of variables, functions, labels and objects within a program is shown here.

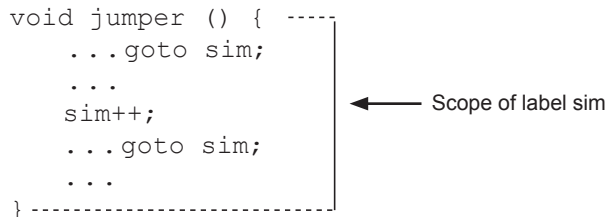
Scope of variables in statement blocks:



❑ Scope of formal arguments of functions:



❑ Scope of labels:



- ❑ **Scope in class declaration (scope of declaration):** The portion of the program in which a declaration can be applied is called the scope of that declaration. In a procedure a name is said to be local to the procedure if it is in the scope of declaration within the procedure, otherwise the name is said to be non-local.

Scope of object fields and methods:

```

class X { -----
    public:
        void A()
        {
            m = 1;
        }
    private:
        int m;
        ...
} -----

```

← Scope of variable m and method A

8. Differentiate between lexical scope and dynamic scope.

Ans: The differences between lexical scope and dynamic scope are given in Table 10.1.

Table 10.1 Difference between lexical and dynamic scope

Lexical Scope	Dynamic Scope
● The binding of name occurrences to declarations is done statically at compile time. ● The structure of the program defines the binding of variables. ● A free variable in a procedure gets its value from the environment in which the procedure is defined.	● The binding of name occurrences to declarations is done dynamically at run time. ● The binding of variables is defined by the flow of control at the run time. ● A free variable gets its value from where the procedure is called.

9. Explain error detection and recovery in lexical phase, syntactic phase, and semantic phase.

Ans: The classification of errors is given in Figure 10.8.

These errors should be detected during different phases of compiler. Error detection and recovery is one of the main tasks of a compiler. The compiler scans and compiles the entire program, and errors detected during scanning need to be recovered as soon as they are detected.

Usually, most of the errors are encountered during syntax and semantic analysis phase. Every phase of a compiler expects the input to be in particular format, and an error is returned by the

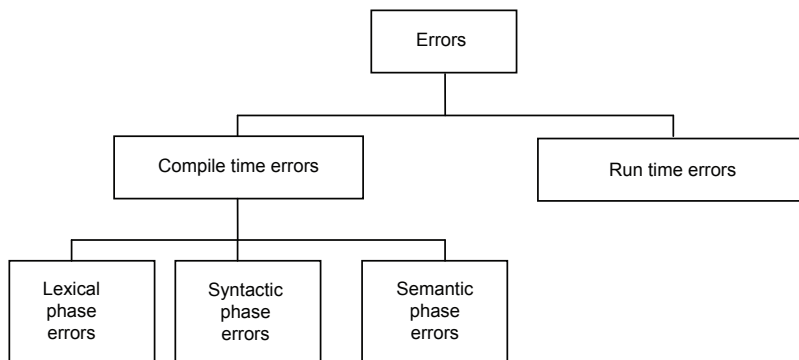


Figure 10.8 Classification of Errors

compiler whenever the input is not in the required format. On detection of an error, the compiler scans some of the tokens ahead of the point of error occurrence. A compiler is said to have better error-detection capability if it needs to scan only a few numbers of tokens ahead of the point of error occurrence.

A good error detection scheme reports errors in an intelligible manner and should possess the following properties.

- ❑ The error message should be easily understandable.
- ❑ The error message should be produced in terms of original source program and not in any internal representation of the source program. For example, each error message should have a line number of the source program associated with it.
- ❑ The error message should be specific and properly localize the error. For example, an error message should be like, “A is not declared in function `sum`” and not just “missing declaration”.
- ❑ The same error message should not be produced again and again, that is, there is no redundancy in the error messages.

Error detection and recovery in lexical Phase: The errors where the remaining input characters do not form any token of the language are detected by the lexical phase of compiler. Typical lexical phase errors are spelling errors, appearance of illegal characters and exceeding length of identifier or numeric constant.

Once an error is detected, the lexical analyzer calls an error recovery routine. The simplest error recovery routine skips the erroneous characters in the input until the lexical analyzer finds a synchronizing token. But this scheme causes the parser to have a deletion error, which would result in several difficulties for the syntax analysis and for the rest of the phases.

The ability of lexical analyzer to recover from errors can be improved by making a list of legitimate tokens (in the current context) which are accessible to the error recovery routine. With the help of this list, the error recovery routine can decide whether the remaining input characters match with a synchronizing token and can be treated as that token.

Error detection and recovery in syntactic phase: The errors where the token stream violates the syntax of the language and the parser does not find any valid move from its current configuration are detected during the syntactic phase of the compiler. The $LL(1)$ and $LR(1)$ parsers have valid prefix property capability, that is, they report an error as soon as they read an input character which is not a valid continuation of the previous input prefix. In this way, these parsers reduce the amount of erroneous output to be passed to next phases of the compiler.

To recover from these errors, *panic mode recovery* scheme or *phrase level recovery* scheme (discussed in chapter 04) can be used.

Error detection and recovery in semantic phase: The language constructs that have the right syntactic structure but have no meaning to the operation involved are detected during semantic analysis phase. Undeclared names, type incompatibilities and mismatching of actual arguments with formal arguments are the main causes of semantic errors. When an undeclared name is encountered first time, a symbol table entry is created for that name with an attribute that is suitable to the current context.

For example, if semantic phase detects an error like “missing declaration of A in function `sum`”, then a symbol table entry is created for A with an attribute that is suitable to the current context. To indicate that an attribute has been added to recover from an error and not in response to the declaration of A, a flag is set in the A symbol table record.

Multiple-Choice Questions

1. Which of the following is not true in context of a symbol table?
 - (a) It is a compile time data structure.
 - (b) It maps name into declarations.
 - (c) It does not help in error detection and recovery.
 - (d) It contains formal parameter list and return type of each function and procedure.
2. The information in the symbol table is entered during _____.
 - (a) Lexical analysis
 - (b) Syntax analysis
 - (c) Both (a) and (b)
 - (d) None of these
3. Which of these operations can be performed on a symbol table?
 - (a) Insert
 - (b) Lookup
 - (c) begin_scope and end_scope
 - (d) All of these
4. Which of the following data structure is not used to implement symbol tables?
 - (a) Linear list
 - (b) Hash table
 - (c) Binary search tree
 - (d) AVL tree
5. Which of the following is not true for scope representation in symbol table?
 - (a) Declarations have same scope in different languages.
 - (b) The scope of a name is a single subroutine in FORTRAN.
 - (c) Symbol table keeps different declaration of the same identifier distinct.
 - (d) In ALGOL, the scope of a name is the section or procedure in which it declared.
6. Which of the following is not true for error detection and recovery?
 - (a) Error detection and recovery is the main task of the compiler.
 - (b) Most of the errors are detected during lexical phase.
 - (c) A compiler returns an error, if the input is not in the required format.
 - (d) None of these

Answers

1. (c) 2. (b) 3. (c) 4. (b) 5. (a) 6. (c) 7. (b)